UNIVERSIDAD SIGLO 21
La educación evoluciona

Seminario de Práctica de Informática

ACTIVIDAD PRÁCTICA 3

SITRAC-MET

Desarrollo del prototipo Java para la gestión y trazabilidad de instrumentos de medición y calibraciones

Alumno: Ricardo Vidal

Materia: Seminario de Práctica de Informática

Actividad: AP3

Fecha: Octubre de 2026

Repositorio GitHub: https://github.com/rickynorby-dot/VIDAL-RICARDO-AP1

Índice

1. Continuidad del proyecto y alcance del AP3

2. Explicación del desarrollo en Java

2.1. Organización del código y correspondencia con el diseño

2.2. Sintaxis, tipos de datos, objetos y constructores

3. Aplicación de los pilares de la programación orientada a objetos

3.1. Encapsulamiento

3.2. Abstracción

3.3. Herencia

3.4. Polimorfismo

4. Estructuras de control y flujo del programa

5. Estructuras de datos, búsqueda y ordenación

6. Tratamiento y manejo de excepciones

7. Menú de selección y funcionalidades del prototipo

8. Persistencia MySQL y continuidad con el AP2

9. Presentación del desarrollo en Java

10. Compilación y ejecución del prototipo

11. Trazabilidad con la consigna

12. Repositorio GitHub y estructura de entrega

13. Conclusiones

14. Referencias

1. Continuidad del proyecto y alcance del AP3

Este tercer trabajo práctico continúa el desarrollo incremental de SITRAC-MET, sistema definido en el AP1 y profundizado en el AP2. El problema de negocio permanece sin cambios: una empresa industrial necesita centralizar el registro de sus instrumentos de medición, conocer su estado, controlar la vigencia de las calibraciones y conservar la trazabilidad de las asignaciones y devoluciones. En la entrega anterior se establecieron el modelo de análisis y diseño, el esquema relacional MySQL, las consultas SQL y una arquitectura inicial cliente-servidor.

En esta iteración el foco se traslada a la implementación orientada a objetos. El objetivo no es construir todavía la totalidad del sistema definitivo, sino obtener un prototipo operacional que permita ejecutar reglas relevantes del dominio y verificar que el diseño puede transformarse en código Java mantenible. Este enfoque es coherente con el carácter incremental del Proceso Unificado de Desarrollo y con la noción de prototipo operacional planteada por Kendall y Kendall (2011).

El incremento implementado permite registrar instrumentos eléctricos y mecánicos, listarlos y buscarlos, registrar calibraciones, validar si un instrumento puede ser asignado, registrar su devolución y consultar el estado de las asignaciones. La persistencia definitiva continúa siendo MySQL, según lo definido en el AP2; en esta entrega se incorpora además la capa Java de acceso JDBC, manteniendo separadas las responsabilidades de dominio y persistencia.

Aspecto	Decisión adoptada
Lenguaje	Java, empleando clases, enumeraciones, colecciones, excepciones y estructuras de control.
IDE	Apache NetBeans como entorno previsto para edición, compilación y ejecución del prototipo.
Persistencia	MySQL, utilizando JDBC y sentencias preparadas en la capa DAO.
Alcance AP3	Implementar el núcleo orientado a objetos y un menú de consola operacional.
Repositorio	Entrega incremental en GitHub, incorporando código, scripts SQL, diagramas y evidencias.

2. Explicación del desarrollo en Java

La implementación toma como punto de partida el diagrama de clases elaborado en el AP2. En lugar de traducir cada tabla de la base directamente a una clase, se priorizan responsabilidades del dominio. La clase abstracta InstrumentoMedicion concentra el estado y comportamiento común; InstrumentoElectrico e InstrumentoMecanico representan especializaciones; GestorInstrumentos coordina las reglas de negocio; y las entidades Calibracion, Asignacion, Responsable y Sector completan el modelo necesario para el prototipo.

El módulo 3 de la materia propone precisamente transformar el diseño previo en clases Java e incorporar encapsulamiento, herencia, abstracción, polimorfismo, constructores, estructuras de control y excepciones. La solución presentada sigue esa progresión y mantiene la separación de responsabilidades para facilitar el mantenimiento y las próximas iteraciones.

2.1. Organización del código y correspondencia con el diseño

Paquete / clase	Responsabilidad principal
sitracmet.modelo.InstrumentoMedicion	Clase abstracta que encapsula los datos y reglas comunes a todo instrumento.
InstrumentoElectrico / InstrumentoMecanico	Especializaciones que agregan su rango particular y redefinen descripcionTipo().

Calibracion	Representa fecha, resultado, certificado, laboratorio y próximo vencimiento.
Asignacion	Registra entrega, devolución, responsable y condiciones del instrumento.
GestorInstrumentos	Mantiene colecciones y ejecuta búsqueda, ordenación, calibración, asignación y devolución.
sitracmet.excepcion	Excepciones de dominio para condiciones no válidas.
sitracmet.dao	Capa JDBC preparada para persistir instrumentos y calibraciones en MySQL.
SitracMetApp	Clase principal. Presenta el menú y coordina la interacción con el usuario.

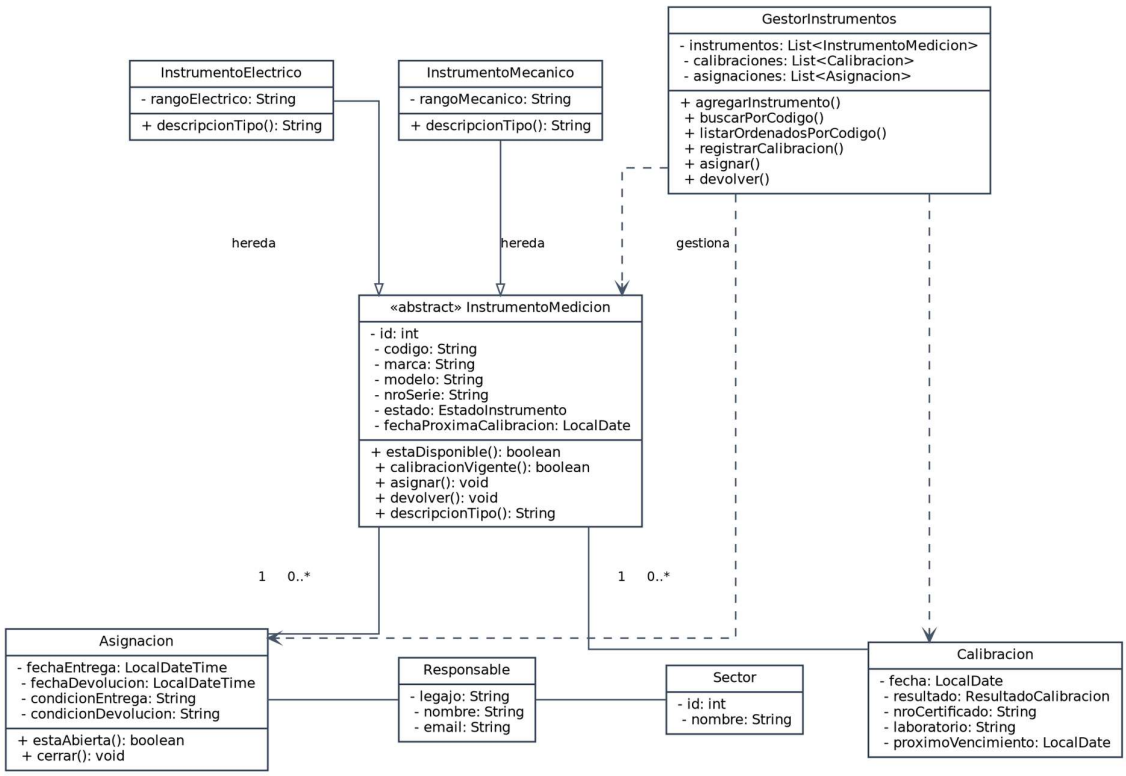


Figura 1. Diagrama de clases implementado para el prototipo AP3 de SITRAC-MET.

2.2. Sintaxis, tipos de datos, objetos y constructores

Se utilizan tipos primitivos y tipos de referencia según la naturaleza del dato. Los identificadores se modelan como int; los textos como String; las fechas de vencimiento mediante LocalDate; las fechas de entrega y devolución mediante LocalDateTime; y los estados mediante enumeraciones, evitando valores libres que puedan generar inconsistencias.

Los objetos se crean mediante constructores que garantizan un estado inicial válido. Por ejemplo, InstrumentoElectrico obliga a informar código, marca, modelo, número de serie, estado, vencimiento de calibración y rango eléctrico. El constructor de la superclase valida identificadores y textos antes de aceptar el objeto.

```
InstrumentoMedicion instrumento = new InstrumentoElectrico(
    1, "INS-001", "Fluke", "179", "FL179-001",
    EstadoInstrumento.DISPONIBLE, LocalDate.now().plusMonths(6),
```

```
   "0-1000 V");
```

3. Aplicación de los pilares de la programación orientada a objetos

La implementación no incorpora los pilares de POO como elementos aislados, sino como decisiones que resuelven necesidades concretas del dominio. La combinación de estos mecanismos permite reducir duplicación, controlar el estado de los objetos y extender el sistema sin alterar innecesariamente el código existente (Booch, 1996; Deitel & Deitel, 2016).

3.1. Encapsulamiento

Los atributos de las entidades se declaran private y no se exponen para modificación directa. Las transiciones de estado se realizan mediante operaciones con intención de negocio, como asignar(), devolver(), enviarACalibracion() y registrarVencimientoCalibracion(). De esta forma, el resto del programa no necesita conocer cómo se representa internamente el estado del instrumento.

```
private EstadoInstrumento estado;

public void asignar() {
   this.estado = EstadoInstrumento.ASIGNADO;
}

public void devolver() {
   this.estado = EstadoInstrumento.DISPONIBLE;
}
```

3.2. Abstracción

InstrumentoMedicion se declara abstracta porque representa las características comunes de cualquier instrumento, pero no corresponde crear instancias genéricas sin un tipo concreto. La clase define el método abstracto descripcionTipo(), que obliga a cada especialización a expresar su comportamiento particular.

```
public abstract class InstrumentoMedicion {
   ...
   public abstract String descripcionTipo();
}
```

3.3. Herencia

InstrumentoElectrico e InstrumentoMecanico extienden InstrumentoMedicion. Ambas clases reutilizan identificación, marca, modelo, serie, estado y reglas de calibración, y agregan únicamente la información propia de su especialidad. Esto evita duplicar código y mantiene una jerarquía coherente con el diseño del AP2.

```
public class InstrumentoElectrico extends InstrumentoMedicion { ... }
public class InstrumentoMecanico extends InstrumentoMedicion { ... }
```

3.4. Polimorfismo

El gestor mantiene una sola colección List<InstrumentoMedicion> que puede contener objetos eléctricos y mecánicos. Al invocar descripcionTipo() o toString(), Java selecciona en tiempo de ejecución la implementación correspondiente al objeto real. De este modo, el menú puede tratar todos los instrumentos de forma uniforme sin preguntar continuamente por su clase concreta.

```
for (InstrumentoMedicion instrumento : instrumentos) {
   System.out.println(instrumento); // ejecuta el comportamiento de la subclase
}
```

4. Estructuras de control y flujo del programa

El flujo principal se implementa mediante un ciclo while que mantiene activo el menú hasta que el usuario selecciona la opción de salida. Dentro del ciclo, una estructura switch dirige la ejecución a la operación elegida. Las reglas de negocio se validan con estructuras if y las colecciones se recorren con for. Estas decisiones permiten expresar de manera directa las alternativas y repeticiones requeridas por la consigna.

Estructura	Uso en SITRAC-MET
while	Mantiene operativo el menú mientras la variable continuar sea verdadera.
switch	Selecciona altas, consultas, calibraciones, asignaciones, devoluciones o salida.
if / else	Valida disponibilidad, vigencia de calibración, duplicados y colecciones vacías.
for	Recorre instrumentos y asignaciones; también se utiliza en la búsqueda secuencial.
while interno	Se utiliza en el algoritmo de ordenación por inserción para desplazar elementos.

```
while (continuar) {
   int opcion = leerEntero("Seleccione una opción: ");
   switch (opcion) {
      case 3 -> listarInstrumentos();
      case 6 -> asignarInstrumento();
      case 0 -> continuar = false;
      default -> System.out.println("Opción fuera de rango.");
   }
}
```

5. Estructuras de datos, búsqueda y ordenación

Para el prototipo se utilizan ArrayList a través de la interfaz List. Esta estructura resulta adecuada porque la cantidad de instrumentos del escenario académico es reducida, permite altas dinámicas y facilita la iteración. Se mantienen colecciones separadas de instrumentos, calibraciones y asignaciones, evitando mezclar responsabilidades.

Aunque los algoritmos de búsqueda y ordenación son opcionales en la consigna, se incorporan para demostrar su aplicación. La búsqueda por código es secuencial y tiene un costo O(n). El listado ordenado utiliza ordenación por inserción sobre una copia de la lista, con costo $O(n^2)$ en el peor caso. Esta decisión es apropiada para un prototipo de bajo volumen; en la versión persistente, búsquedas masivas deberán resolverse principalmente mediante índices y consultas SQL.

```
// Búsqueda secuencial
for (InstrumentoMedicion instrumento : instrumentos) {
    if (instrumento.getCodigo().equalsIgnoreCase(codigo)) {
        return instrumento;
    }
}

// Ordenación por inserción
for (int i = 1; i < copia.size(); i++) {
    InstrumentoMedicion actual = copia.get(i);
    int j = i - 1;
    while (j >= 0 && comparador.compare(copia.get(j), actual) > 0) {
        copia.set(j + 1, copia.get(j));
        j--;
    }
    copia.set(j + 1, actual);
}
```

6. Tratamiento y manejo de excepciones

Las excepciones se utilizan para separar los casos normales de las situaciones que impiden completar una operación. Se definieron excepciones propias del dominio y, además, el menú captura errores de formato. La aplicación informa el problema y continúa funcionando, evitando que un dato incorrecto finalice abruptamente el programa.

Excepción	Situación controlada
CodigoDuplicadoException	Intento de registrar un código de instrumento ya existente.
InstrumentoNoEncontradoException	Búsqueda, asignación o devolución de un código inexistente.
InstrumentoNoDisponibleException	Intento de asignar un instrumento cuyo estado no permite la entrega.
CalibracionVencidaException	Intento de asignar un instrumento sin calibración vigente.
NumberFormatException / DateTimeParseException	Entrada del usuario con número o fecha inválidos.
SQLException	Errores de acceso a MySQL dentro de la capa DAO.

```
if (!instrumento.estaDisponible()) {
    throw new InstrumentoNoDisponibleException(codigo);
}
if (!instrumento.calibracionVigente()) {
    throw new CalibracionVencidaException(codigo);
}
```

7. Menú de selección y funcionalidades del prototipo

La clase SitracMetApp presenta un menú de consola que permite recorrer las operaciones implementadas sin requerir una interfaz gráfica. El menú funciona como punto de entrada del prototipo y demuestra creación de objetos, estructuras de control, validaciones y tratamiento de excepciones.

Opción	Funcionalidad

1	Registrar instrumento eléctrico.
2	Registrar instrumento mecánico.
3	Listar instrumentos ordenados por código.
4	Buscar instrumento por código.
5	Registrar calibración.
6	Asignar instrumento a un responsable.
7	Registrar devolución.
8	Consultar asignaciones.
0	Finalizar el programa.

La regla más importante del incremento se aplica al asignar: el instrumento debe existir, encontrarse DISPONIBLE y poseer una calibración cuya fecha de vencimiento no sea anterior a la fecha actual. Si alguna condición no se cumple, la asignación no se registra y se informa la causa al usuario.

8. Persistencia MySQL y continuidad con el AP2

La persistencia definida en el AP2 se mantiene sin modificar: la base sitrac_met se implementa en MySQL y contiene entidades para instrumento, tipo de instrumento, calibración, asignación, responsable, sector, laboratorio, rol y usuario. Los scripts 01_schema.sql, 02_datos_prueba.sql y 03_consultas.sql se incluyen nuevamente en el repositorio para asegurar una entrega incremental.

En Java se incorpora la capa dao. ConexionBD obtiene la URL, usuario y contraseña desde variables de entorno y abre la conexión mediante DriverManager. InstrumentoDAO y CalibracionDAO utilizan PreparedStatement, lo que permite separar el SQL de la lógica del dominio y evita concatenar directamente datos ingresados por el usuario. Para ejecutar la persistencia real es necesario agregar MySQL Connector/J a las librerías del proyecto en NetBeans.

```
String url = System.getenv().getOrDefault(
    "SITRAC_DB_URL", "jdbc:mysql://localhost:3306/sitrac_met");
return DriverManager.getConnection(url, usuario, clave);
```

Durante la verificación de este AP3 se ejecutó el núcleo de dominio y el menú sin requerir una base externa, de modo de comprobar de forma reproducible la lógica POO. La capa JDBC compila junto con el resto del proyecto y queda preparada para conectarse al esquema MySQL del AP2 cuando se configure el driver y las credenciales. Esta separación también evita publicar contraseñas reales en GitHub.

9. Presentación del desarrollo en Java

La presentación del desarrollo se realiza mediante el código fuente incluido en GitHub y una evidencia de ejecución. La siguiente captura corresponde a una corrida real del prototipo compilado en Java. Se observa el menú, el listado polimórfico de instrumentos, la búsqueda por código, una asignación válida, el rechazo de un instrumento con calibración vencida y el posterior cierre de la asignación mediante devolución.



Figura 2. Evidencia de ejecución del menú y validaciones del prototipo SITRAC-MET.

10. Compilación y ejecución del prototipo

Se verificó la compilación completa de las clases con JDK 21 y la ejecución de la clase principal sitracmet.app.SitracMetApp. No se registraron errores de compilación. La ejecución finalizó de manera controlada luego de recorrer operaciones de consulta, asignación, tratamiento de una calibración vencida y devolución.

```
javac -encoding UTF-8 -d out $(find src -name "*.java")
java -Dfile.encoding=UTF-8 -cp out sitracmet.app.SitracMetApp
```

Verificación	Resultado
Compilación de todas las clases	Correcta. Sin errores de sintaxis o tipos.
Inicio y visualización del menú	Correcta.
Listado polimórfico	Correcto: eléctricos y mecánicos se recorren como InstrumentoMedicion.
Búsqueda secuencial	Correcta para INS-001.
Asignación con calibración vigente	Correcta.
Asignación con calibración vencida	Rechazada mediante CalibracionVencidaException.
Devolución y cierre de asignación	Correctos.
Finalización de aplicación	Controlada mediante opción 0.

11. Trazabilidad con la consigna

La siguiente matriz permite comprobar que cada característica obligatoria solicitada en el enunciado posee una implementación concreta en el prototipo.

Requisito de la consigna	Implementación / evidencia
Sintaxis y tipos de datos	Clases Java, int, boolean, String, LocalDate, LocalDateTime y enum.
Estructuras de control	while, switch, if y for en SitracMetApp y GestorInstrumentos.
Manejo de excepciones	Excepciones de dominio más errores de formato y SQLException.
Encapsulamiento	Atributos privados y operaciones controladas sobre el estado.
Herencia	InstrumentoElectrico e InstrumentoMecanico extienden InstrumentoMedicion.
Polimorfismo	List<InstrumentoMedicion> y redefinición de descripcionTipo().
Abstracción	InstrumentoMedicion es abstracta y define comportamiento común.
Menú de selección	SitracMetApp presenta opciones 0 a 8.
Objetos y constructores	Creación de instrumentos, responsables, sectores, calibraciones y asignaciones.
Estructuras de datos	ArrayList/List para las colecciones del dominio.
Ordenación y búsqueda	Ordenación por inserción y búsqueda secuencial.
Persistencia MySQL	Scripts del AP2 más capa DAO/JDBC incluida en AP3.
Programa compila y ejecuta	Compilación y recorrido de menú verificados; evidencia incluida.

12. Repositorio GitHub y estructura de entrega

La versión completa del prototipo, el código fuente y los archivos asociados se entregan en el siguiente repositorio incremental:

https://github.com/rickynorby-dot/VIDAL-RICARDO-AP1

La estructura propuesta para esta tercera entrega es:

```
SITRAC-MET-AP3/
 ├── src/
|   └── sitracmet/
|       ├── app/
|       ├── modelo/
|       ├── servicio/
|       ├── excepcion/
|       └── dao/
 ├── sql/
 ├── diagramas/
 ├── evidencias/
 ├── docs/
 └── README.md
```

No se incluyen contraseñas reales de MySQL. Las credenciales deben definirse localmente mediante variables de entorno antes de habilitar la conexión JDBC.

13. Conclusiones

El AP3 permitió transformar el diseño realizado en las etapas anteriores en un prototipo Java ejecutable. La estructura implementada conserva la lógica del dominio y demuestra la aplicación concreta de encapsulamiento, abstracción, herencia y polimorfismo, evitando que estos conceptos queden solamente en una definición teórica.

El menú permite comprobar las operaciones principales y sus reglas: los instrumentos pueden registrarse y consultarse, las asignaciones requieren disponibilidad y calibración vigente, y las devoluciones cierran el movimiento correspondiente. La incorporación de excepciones específicas mejora la claridad del flujo cuando una operación no puede realizarse.

La utilización de List y ArrayList facilita la construcción del prototipo, mientras que la búsqueda secuencial y la ordenación por inserción permiten aplicar los contenidos de algoritmos y estructuras de datos. En paralelo, la capa DAO mantiene la continuidad con la base MySQL definida en el AP2 y prepara la siguiente iteración para una integración más completa entre interfaz, lógica y persistencia.

14. Referencias

Booch, G. (1996). Análisis y diseño orientado a objetos con aplicaciones. Addison-Wesley Iberoamericana.

Deitel, P. J., & Deitel, H. M. (2016). Cómo programar en Java. Pearson Educación.

Kendall, K. E., & Kendall, J. E. (2011). Análisis y diseño de sistemas (8.ª ed.). Pearson Educación.

Pressman, R. S. (2010). Ingeniería del software: Un enfoque práctico. McGraw-Hill.

Shaw, M., & Garlan, D. (2015). Software architecture: Perspectives on an emerging discipline. Pearson.